QUESTION-29:

Write a C program to simulate the solution of Classical Process Synchronization Problem .

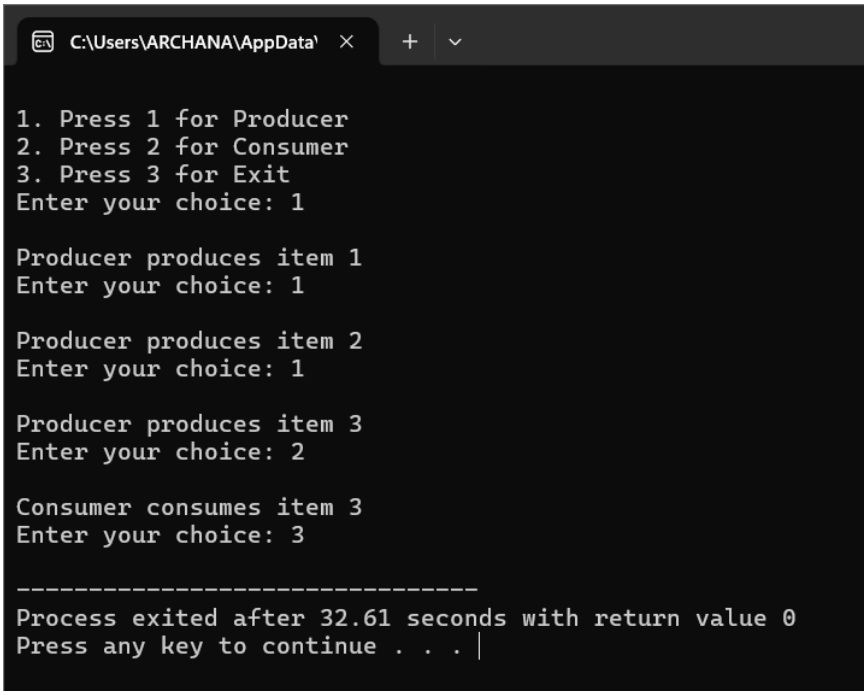
PROGRAM:

```
#include    <stdio.h>
#include    <stdlib.h>
int mutex = 1;
int full = 0;
int empty = 10, x = 0;
void producer() {
    --mutex;
    ++full;
    --empty;
    x++;
    printf("\nProducer produces item %d", x);
    ++mutex;
}
void consumer() {
    --mutex;
    --full;
    ++empty;
    printf("\nConsumer consumes item %d", x);
    x--;
    ++mutex;
}
int main() {
    int n, i;
    printf("\n1. Press 1 for Producer"
           "\n2. Press 2 for Consumer"
           "\n3. Press 3 for Exit");
    #pragma omp critical
```

```
for (i = 1; i > 0; i++) {
    printf("\nEnter your choice: ");
    scanf("%d", &n);
    switch (n)
    {
        case 1:
            if ((mutex == 1) && (empty != 0))
            {
                producer();
            }
            else
            {
                printf("Buffer is full!");
            }
            break;
        case 2:
            if ((mutex == 1) && (full != 0))
            {
                consumer();
            }
            else
            {
                printf("Buffer is empty!");
            }
            break;
        case 3:
            exit(0);
            break;
    }
}
```

}

OUTPUT:



QUESTION-30:

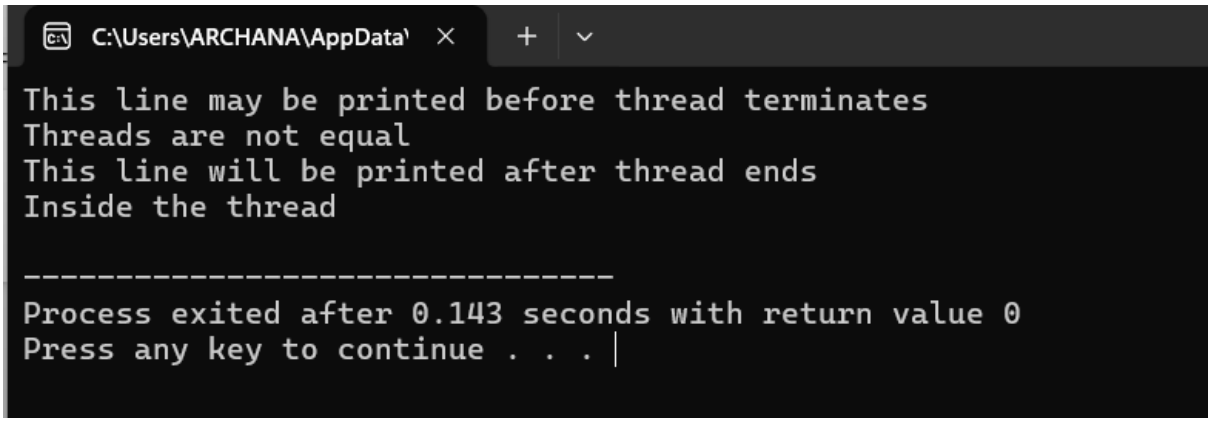
Write C programs to demonstrate the following thread related concepts.

PROGRAM:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
void* func(void* arg)
{
   pthread_detach(pthread_self());
   printf("Inside the thread\n");
   pthread_exit(NULL);
}
void fun()
{
```

```
    pthread_t ptid;
    pthread_create(&ptid, NULL, &func, NULL);
    printf("This line may be printed before thread terminates\n");
    if (pthread_equal(ptid, pthread_self()))
    {
        printf("Threads are equal\n");
    }
    else
        printf("Threads are not equal\n");
    pthread_join(ptid, NULL);
    printf("This line will be printed after thread ends\n");
    pthread_exit(NULL);
}
int main()
{
    fun();
    return 0;
}
```

OUTPUT:



QUESTION-31:

Construct a C program to simulate the First in First Out paging technique of memory management.

PROGRAM:

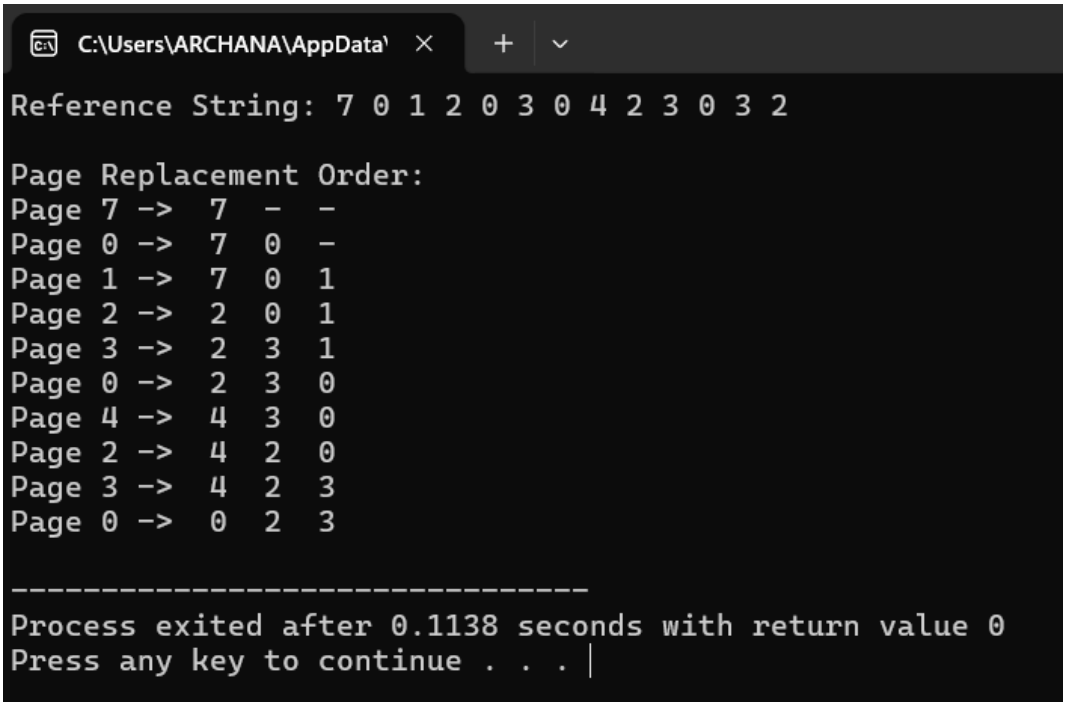
```
#include <stdio.h>
#define MAX_FRAMES 3 // Maximum number of frames in memory
void printFrames(int frames[], int n) {
    for (int i = 0; i < n; i++) {
        if (frames[i] == -1) {
            printf(" - ");
        } else {
            printf(" %d ", frames[i]);
        }
    }
    printf("\n");
}
int main() {
    int referenceString[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2};
    int n = sizeof(referenceString) / sizeof(referenceString[0]);
    int frames[MAX_FRAMES];
    int framePointer = 0; // Points to the current frame to be replaced

    for (int i = 0; i < MAX_FRAMES; i++) {
        frames[i] = -1; // Initialize all frames to -1 (indicating empty)
    }
    printf("Reference String: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", referenceString[i]);
    }
    printf("\n\n");
    printf("Page Replacement Order:\n");
    for (int i = 0; i < n; i++) {
        int page = referenceString[i];
```

```
        int pageFound = 0;

        // Check if the page is already in memory
        for (int j = 0; j < MAX_FRAMES; j++) {
            if (frames[j] == page) {
                pageFound = 1;
                break;
            }
        }
        if (!pageFound) {
            printf("Page %d -> ", page);
            frames[framePointer] = page;
            framePointer = (framePointer + 1) % MAX_FRAMES;
            printFrames(frames, MAX_FRAMES);
        }
    }
    return 0;
}
```

OUTPUT:



QUESTION-32:

Construct a C program to simulate the Least Recently Used paging technique of memory management.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_FRAMES 3
void printFrames(int frames[], int n) {
    for (int i = 0; i < n; i++) {
        if (frames[i] == -1) {
            printf(" - ");
        } else {
            printf(" %d ", frames[i]);
        }
    }
    printf("\n");
}
int main() {
    int frames[MAX_FRAMES];
    int usageHistory[MAX_FRAMES]; // To store the usage history of pages
    for (int i = 0; i < MAX_FRAMES; i++) {
        frames[i] = -1; // Initialize frames to -1 (empty)
        usageHistory[i] = 0; // Initialize usage history
    }
    int pageFaults = 0;
    int referenceString[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2};
    int n = sizeof(referenceString) / sizeof(referenceString[0]);
    printf("Reference String: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", referenceString[i]);
```

```
}
printf("\n\n");
printf("Page Replacement Order:\n");
for (int i = 0; i < n; i++) {
    int page = referenceString[i];
    int pageFound = 0;
    // Check if the page is already in memory (a page hit)
    for (int j = 0; j < MAX_FRAMES; j++) {
        if (frames[j] == page) {
            pageFound = 1;
            // Update the usage history by incrementing other pages
            for (int k = 0; k < MAX_FRAMES; k++) {
                if (k != j) {
                    usageHistory[k]++;
                }
            }
            usageHistory[j] = 0; // Reset the usage counter for the used page
            break;
        }
    }
    if (!pageFound) {
        printf("Page %d -> ", page);
        // Find the page with the maximum usage counter (least recently used)
        int lruPage = 0;
        for (int j = 1; j < MAX_FRAMES; j++) {
            if (usageHistory[j] > usageHistory[lruPage]) {
                lruPage = j;
            }
        }
        int replacedPage = frames[lruPage];
```

```
            frames[lruPage] = page;
            usageHistory[lruPage] = 0;
            if (replacedPage != -1) {
                printf("Replace %d with %d: ", replacedPage, page);
            } else {
                printf("Load into an empty frame: ");
            }
            printFrames(frames, MAX_FRAMES);
            pageFaults++;
        }
    }
    printf("\nTotal Page Faults: %d\n", pageFaults);
    return 0;
}
```

OUTPUT:

